Computer Architecture Lecture 5
August 12th 2024

CS23BTECH11035 Sai Manjunath Mathapathi
CS23BTECH11058 Sri Ujwal Srinivas Varma Gunturi

Instruction Formats

Every instruction in a RISC-V base integer ISA is of size 32 bits or 4 bytes long.

Each instruction is divided into a series of bits known as **fields**. Instructions come under different Instruction formats based on which of these fields are present in the 32 bits of the instruction.

Why is the size of each instruction 32 bits long? Because RISC-V is aimed to be simple and making every instruction 32 bits makes the hardware design easy. Making each instruction 64 bits or more just increases the size of the program, and making each instruction 16 bits or less will make it difficult to represent all the registers in an instruction.

Types of Fields

Operation Code (opcode)

The opcode tells what operation is going to be performed. Every operation has a unique opcode, or at least they differ in additional opcode fields.

The additional opcode fields are used to differentiate between instructions that have the same opcode. *funct3* and *funct7* are additional opcode fields in which *3* and *7* represent the number of bits used by each, respectively.

The size of the opcode in every instruction format is the same, and it is equal to 7 bits.

Destination Register Field (rd)

This field represents the register in which the result of the operation should be stored. Since we have 32 registers, we need at least 5 bits to represent all the registers uniquely, so the size of this field is 5 bits.

Operand 1 Register (rs1)

This field represents the first operand of an instruction, and we need 5 bits to represent it.

Operand 2 Register (rs2)

This field represents the second operand of an instruction, and we need 5 bits to represent it.

Immediate Value (imm)

This field is present in instructions that have immediate values in them. Since in this type of instruction, we do not require a second operand register, the 5 bits used to represent this second operand register are added to the immediate field. Additionally, the opcode field *funct7* is not present.

The size of the immediate field is 12 bits.

Types of Instruction Formats

R (Register) Format

This instruction format contains instructions that require three register operands, i.e., *rs1*, *rs2*, and *rd*. This instruction format contains *funct3* and *funct7* additional opcode fields. This instruction format requires both additional opcode fields because there are many operations that require three register operands.

FUNCT7	RS2	RS1	FUNCT3	RD	OPCODE
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Instructions that use this format are: *add, sub, and, or, xor, sll, srl, sra, slt, sltu*

Example: add x8, x5, x6 format is add rd, rs1, rs2

FUNCT7	RS2	RS1	FUNCT3	RD	OPCODE
0000000	00110	00101	000	01000	0110011

Here, RD is x8 (01000), RS1 is x5 (00101), and RS2 is x6 (00110).

I (Immediate) Format

This type of format is followed by the instructions that do not have a second source operand register, and the additional opcode field *funct7* is not present in this format. This allows 12-bits to be used for the immediate value.

IMMEDIATE	RS1	FUNCT3	RD	OPCODE
12 bits	5 bits	3 bits	5 bits	7 bits

The bits in the immediate field are interpreted as 2's complement binary numbers, so the range of possible values in the immediate is from -2048 to 2047.

Instructions that use this format are: *addi, ori, andi, xori, slli, srli, srai, slti, sltiu, lb, lw, lh, ld, lbu, lwu, lhu, ecall, ebreak, jalr*

All variants of Load instructions (ld, lw, lh, lb, lwu, lhu, lbu) also come under this format because they only have one source register and one destination register operands, and a third operand being the offset (immediate value) from the first operand.

Example: addi x8, x5, 36 format is addi rd, rs1, imm

IMMEDIATE	RS1	FUNCT3	RD	OPCODE
000000100100	00101	000	01000	0010011

In this case, x5 (00101) is RS1, 36 (000000100100) is the immediate value and x8 (01000) is RD.

S (Store) Format

This instruction format is followed by the store instructions. While being similar to load instruction in the number of operands, we do not have a destination register operand in the store instruction but two source operands are present. In a way, it is similar to load instructions, but the starting 5 bits of the immediate field are stored in the place where *rd* used to be present. The reason is that keeping the position of register fields same in every instruction format makes hardware simpler.

IMM[11:5]	RS2	RS1	FUNCT3	IMM[4:0]	OPCODE
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

The syntax for store is slightly different: *"sd rs2, offset(rs1)"*. *rs2* comes before *rs1*. Instructions that use this format are: *sb, sw, sh, sd*

Example: sd x8, 8(x3)
RS2 is x8 (01000), RS1 is x3 (00011) and the immediate offset is 8 (000000001000) in this example.

IMM[11:5]	RS2	RS1	FUNCT3	IMM[4:0]	OPCODE
0000000	01000	00011	011	01000	0100011

B (Branch) Format

This instruction format is followed by the branch instructions. It is similar to the S-type instruction format. The immediate value in this format is 13 bits long, and the least significant bit of the immediate value is ignored because the size of the instructions is a multiple of 4, so the 2 least significant bits can be ignored. However, only the least significant bit is ignored to support devices that use compressed instructions (16 bits long). Since the immediate value will

always have the LSB as 0, we ignore it during instruction representation. This allows to have a longer range supported for the branch instructions.

IMM[12]—IMM[10:5]	RS2	RS1	FUNCT3	IMM[4:1]—IMM[11]	OPCODE
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

The LSB bit of the immediate is ignored, and in place of this, the 12th bit of the immediate is placed. The 11th bit is not placed anywhere else because it disturbs the relative position of the other 4 bits of the immediate with the S-type instruction format.

Instructions that follow this instruction format are: *beq, bne, blt, bge, bltu, bgeu*

Example: beq x8, x5, Label1

Here, we are assuming Label1 offsets the program counter by +32 bytes. x8 (01000) is RS1, x5 (00101) is RS2, and Label1 is the immediate offset (+32) in this case.

The 2's complement representation of +32 in 13-bits is: 0000000100000 (IMM[12:0]). We extract bits from appropriate positions and use them as per the instruction enconding.

IMM[12]—IMM[10:5]	RS2	RS1	FUNCT3	IMM[4:1]—IMM[11]	OPCODE
0000001	00101	01000	000	00000	1100011

Relation Between Machine Code and Assembly Code

The assembly code and machine code are one-to-one equivalent, considering we are not using pseudo-instructions. For every instruction the programmer has written, it is converted into one machine code instruction.

Instruction to Machine Code Conversion Example

Convert this instruction to machine code: **add x9, x20, x21**

The opcode for *add* instruction is 51, and additional opcode fields are all zero. The first operand (*rs1*) is x20 register, and the registers are represented by their index in machine code. The second operand (*rs2*) is x21 register. The destination register is x9.

FUNCT7	RS2	RS1	FUNCT3	RD	OPCODE
0	21	20	0	9	51
0000000	**10101**	**10100**	**000**	**01001**	**0110011**

Converting the result into hexadecimal for convenience:

$$0000\ 0001\ 0101\ 1010\ 0000\ 0100\ 1011\ 0011$$

Resulting in:

$$(015A04B3)_{16}$$